

ETPNav: Evolving Topological Planning for Vision-Language Navigation in Continuous Environments

Dong An¹, Hanqing Wang², Wenguan Wang³, *Senior Member, IEEE*, Zun Wang⁴, Yan Huang⁵, Keji He⁶,
and Liang Wang⁷, *Fellow, IEEE*

Abstract—Vision-language navigation is a task that requires an agent to follow instructions to navigate in environments. It becomes increasingly crucial in the field of embodied AI, with potential applications in autonomous navigation, search and rescue, and human-robot interaction. In this paper, we propose to address a more practical yet challenging counterpart setting - vision-language navigation in continuous environments (VLN-CE). To develop a robust VLN-CE agent, we propose a new navigation framework, ETPNav, which focuses on two critical skills: 1) the capability to abstract environments and generate long-range navigation plans, and 2) the ability of obstacle-avoiding control in continuous environments. ETPNav performs online topological mapping of environments by self-organizing predicted waypoints along a traversed path, without prior environmental experience. It privileges the agent to break down the navigation procedure into high-level planning and low-level control. Concurrently, ETPNav utilizes a transformer-based cross-modal planner to generate navigation plans based on topological maps and instructions. The plan is then performed through an obstacle-avoiding controller that leverages a trial-and-error heuristic to prevent navigation from getting stuck in obstacles. Experimental results demonstrate the effectiveness of the proposed method. ETPNav yields more than 10% and 20% improvements over prior state-of-the-art on R2R-CE and RxR-CE datasets, respectively.

Index Terms—Vision-language navigation, topological map, obstacle avoidance.

Manuscript received 14 April 2023; revised 15 January 2024; accepted 6 April 2024. Date of publication 9 April 2024; date of current version 6 June 2025. This work was supported in part by the National Key R&D Program of China under Grant 2022ZD0117900, in part by the National Natural Science Foundation of China under Grant 62236010, Grant 62276261, Grant 62322607 and Grant 62372405, and in part by CCF-Tencent Open Fund. Recommended for acceptance by Associate Editors P. Xu, S. Bai, B. Zhou, D. Clifton, A. Vedaldi, M. van der Schaar, and L. Van Gool (*Corresponding authors: Wenguan Wang; Yan Huang.*)

Dong An, Yan Huang, Keji He, and Liang Wang are with the Center for Research on Intelligent Perception and Computing (CRIPAC), National Laboratory of Pattern Recognition (NLPR), School of Future Technology and School of Artificial Intelligence, University of Chinese Academy of Sciences, Beijing 101408, China (e-mail: dong.an@cripac.ia.ac.cn; yhuang@nlpr.ia.ac.cn; keji.he@cripac.ia.ac.cn; wangliang@nlpr.ia.ac.cn).

Hanqing Wang is with the Beijing Institute of Technology, Beijing 100811, China (e-mail: hanqingwang@bit.edu.cn).

Wenguan Wang is with the Zhejiang University, Hangzhou, Zhejiang 310027, China (e-mail: wenguanwang.ai@gmail.com).

Zun Wang is with the Australian National University, Canberra, ACT 2601, Australia (e-mail: zun.wang@anu.edu.au).

Our code is available at <https://github.com/MarSaKi/ETPNav>.

Digital Object Identifier 10.1109/TPAMI.2024.3386695

I. INTRODUCTION

GIVEN a natural language instruction, the task of vision-language navigation (VLN) [1] requires an agent to interpret and follow the instruction to reach the target location. This task has been well-studied over the past few years [2], [3], [4], [5], however, the majority of works focus on the discrete VLN setting. This setting simplifies navigation as traversing on a predefined graph of an environment, which significantly narrows down the possible locations of the agent and target. Recognizing this cannot reflect the challenges of a deployed system encountered in a real environment, Krantz et al. [6] introduce VLN in continuous environments (VLN-CE), which discards the strong graph assumption, instead, requires the agent to navigate on a 3D mesh freely with low-level actions.

So far, VLN-CE has been shown far more difficult than VLN, with a few published works revealing episode success rates less than half of those reported in VLN. Early efforts for VLN-CE are end-to-end trained systems to directly predict low-level actions (or waypoints) from language and observations [6], [7], [8]. This scheme can be challenged by joint learning of navigation and language grounding in a long-horizon task, thus leading to a lower performance compared to VLN. Recently, there has been an emerging trend towards modular waypoint-based approaches [9], [10], [11] that divide the complex task into waypoint generation, subgoal planning, and navigation control. Concretely, in each decision loop, the agent uses a pre-trained network to predict several nearby candidate waypoints, and then performs cross-modal grounding to select a subgoal from the waypoints. After that, a controller drives the agent to reach the selected subgoal with low-level actions. Overall, this modular pipeline simplifies policy learning and closes the performance gap between VLN-CE and VLN.

Despite the progress, we find these waypoint-based methods still have drawbacks in three aspects. *First*, the predicted waypoints are still local and constrained to a nearby area of the agent, which are insufficient to capture the global environment layouts and may hinder the agent's long-range planning capacity. For example, to backtrack to a previous remote location for past decision correction, the agent has to run multiple plan-control flows which can introduce unstable accumulation bias. *Second*, the key design choices for waypoint prediction have not been well-studied. One representative predictor [9] takes RGBD

as inputs, but whether or not the semantic-level RGB inputs are valid remains unknown, since it is only tasked with inferring spatial accessibility. *Third*, different types of controllers (heuristic [8], [9], map-based [10], [12], learning-based [13], [14]) are proposed for waypoint-reaching, but their robustness to obstacles remains unknown. As a result, an obstacle-intolerant controller can cause the agent to get stuck in obstacles, leading to navigation failure.

To address the above problems, we propose a hierarchical navigation framework powered by topological (topo) maps, and a low-level controller. Topo maps, partially inspired by cognitive science [15], typically depict environments as low-dimensional graph representations with nodes for places and edges for reachability. They can efficiently capture environment layouts and long-range navigation dependency, thereby easing the agent to make long-range goal plans, such as planning a shortest path within the map to reach a remote location. But what makes our topo maps novel is that they are constructed via online self-organization of predicted waypoints, which are concise and meet the assumption of partial observability in a real environment. Notably, this scheme is greatly distinct from previous VLN literature regarding topo mapping, which requires either predefined graphs [3], [16], [17] or environment pre-exploration [18].

To better capture the environment layouts, we systematically examine our topo map's key design choices, such as waypoint prediction, node density, and node representation. In particular, we find that a depth-only waypoint predictor aids generalization in novel environments, while RGB information may undermine spatial accessibility inferring. Moreover, we explicitly consider the obstacle-avoidance problem in VLN-CE. We find this problem is especially crucial in a more challenging and practical scenario - sliding along obstacles is forbidden, where commonly used controllers [8], [12], [13] can cause navigation to get stuck in obstacles frequently, resulting in a severe performance drop. Accordingly, we propose a new controller via a trial-and-error heuristic to explicitly help the agent escape from deadlocks, nearly eliminating the performance loss caused by sliding-forbidden.

Altogether, we propose a full navigation system for VLN-CE. For each episode, our agent updates a topo map through online self-organization of waypoints predicted so far. The map decomposes the navigation problem into planning and control. Within each decision loop, the agent uses a cross-modal transformer [19], [20] to compute a global navigation plan from the instruction and the topo map. Then, this plan is executed by a robust obstacle-avoiding controller with low-level actions.

Extensive experiments demonstrate the effectiveness of the proposed method, and our system achieves state-of-the-art on two VLN-CE benchmarks (e.g., in test unseen splits, 55 SR and 48 SPL on R2R-CE dataset, 51.21 SR and 41.30 SDTW on RxR-CE dataset). Based on the algorithm described in this paper, we won the CVPR 2022 RxR-Habitat Challenge [11], [21]. In summary, the contributions of this work are four-fold:

- We propose a new topological map-based method for robust navigation planning in VLN-CE. It can efficiently abstract the continuous environments and facilitates the agent's long-range goal planning.

- We investigate the essential design choices for building topological maps through comprehensive experiments, demonstrating that a concise depth-only design is optimal for waypoint prediction.
- We study a practically important but rarely investigated problem in VLN-CE - obstacle avoidance, and propose an effective trial-and-error controller to address the problem.
- The proposed system won the CVPR 2022 RxR-Habitat Challenge and doubled the SDTW of the second-best model. It can serve as a strong baseline for further research on this challenging task.¹

The rest of this paper is organized as follows. In Section II, we give a brief review of the related work. Section III describes the task setup of vision-language navigation in continuous environments and then introduces our proposed method. Experimental results are provided in Section IV. Lastly, we conclude this work in Section V.

II. RELATED WORK

A. Vision-Language Navigation

Learning navigation with language guidance has drawn significant research interest in recent years. R2R [1] and RxR [22] datasets introduce low-level human language instructions and photo-realistic environments for indoor navigation, while Touchdown [23] further extends this task in an outdoor navigation context. Following these works, dialogue-based navigation such as CVDN [24] and HANNA [25], and navigation for remote object-finding such as REVERIE [26] and SOON [27] have been proposed for further research.

Early VLN methods use sequence-to-sequence LSTMs to predict low-level actions [1] or high-level actions from panoramas [28]. Various attention mechanisms [29], [30], [31], [32] are proposed to improve the learning of visual-textual correspondence. Reinforcement learning is also explored to improve policy learning [2], [33], [34], [35]. Different strategies are also investigated to form a more robust navigation policy, such as environment pre-exploration [2], active perception [36], [37], and planning with graph memory [3], [16]. To enhance an agent's generalization ability to novel environments, various data augmentation strategies are studied to mimic new environments [34], [38], [39], [40], [41] or synthesis new instructions [28], [42], [43], [44], [45], [46]. Recently, transformer-based models have shown superior performance thanks to their powerful ability to learn generic multi-modal representations [47], [48], [49]. This scheme is further extended by recurrent agent state [4], [50], [51], episodic memory [5], [52], [53], graph memory [17], [54], [55], [56], [57] and prompt learning [58], [59] that significantly improves sequential action prediction.

Despite the progress, these agents are developed under the discrete VLN setting, which simplifies navigation as traversing on a predefined graph of an environment. In effect, this setup greatly narrows down the possible locations of the agent and target, while ignoring the low-level control problem that arises in a real-world navigation system. As a result, directly

¹Our code is available at <https://github.com/MarSaKi/ETPNav>.

transferring these agents into the real world [60] or continuous environments [6] can cause a severe performance drop.

B. VLN in Continuous Environments

Recognizing the navigation graph assumption cannot reflect the challenges a deployed system would experience in a real environment, Krantz et al. [6] introduce VLN in continuous environments (VLN-CE) - requiring the agent to navigate on a 3D mesh freely with low-level actions (e.g., FORWARD 0.25 m, ROTATE 15°). To benchmark VLN-CE agents, discrete paths in R2R [1] and RxR [22] are transferred to continuous environments through the Habitat Simulator [61].

Initial methods for VLN-CE are end-to-end trained systems to directly predict low-level actions from language and observations [6], [7], [62], but demonstrate a huge performance gap to VLN. Because jointly learning language-grounding and low-level control in a fully end-to-end manner can be difficult and expensive, typically requiring millions of frames of experience [13]. Thus, Krantz et al. [8] propose to decouple the navigation process as subgoal planning and low-level control, where the model predicts a language-conditioned waypoint as subgoal at each step, and then reaches the subgoal with a rotate-then-forward control flow. This idea is further extended with semantic maps for better perception of environments [14], [63], finding increased performance but still far below VLN. One potential reason is that the prediction of language-conditioned waypoints requires the model to learn spatial accessibility inferring and cross-modal reasoning simultaneously, which is difficult and may need massive training [8].

Recently, there has been an emerging trend towards modular waypoint-based approaches [9], [10], [11], [64]. Instead of directly predicting a language-conditioned waypoint, these methods further decouple the subgoal prediction as candidate waypoint generation and subgoal selection. Concretely, within each decision loop, the agent first uses a pre-trained network to predict several nearby candidate waypoints, and then performs cross-modal grounding over the waypoints to select a subgoal. Similarly, the subgoal-reaching is conducted by a follow-up controller. Overall, this modular scheme simplifies policy learning and further closes the gap between VLN-CE and VLN. But the drawback is the local waypoint representations which are insufficient to capture global environment layouts and navigation dependency, leading to the agent's non-ideal long-range planning capability. Meanwhile, the widely used controllers are unaware of obstacles, and we find they can cause navigation to get stuck in obstacles frequently in a practical sliding-forbidden scenario. To address these limitations, we not only propose an online constructed topo map for long-range planning, but also devise an obstacle-avoiding controller.

C. Maps for Navigation

Works on robot navigation have a long tradition of using spatial or topological space representations to aid environmental perception. Researchers have investigated explicit metric spatial representations [65] and their construction using various sensors [66], [67], as well as agent localization with such representations [68], [69]. Modern literature has begun to integrate

spatial representations with semantics, yielding promising results in tasks such as active perception [70], [71], object-goal navigation [72], [73], vision-language navigation [14], [17], [74] and audio-visual navigation [75]. However, these metric maps typically suffer from scalability issues, rendering them unsuitable for long-range navigation tasks such as VLN-CE. For instance, though a large metric map has been shown effective in VLN-CE, it may result in excessive computation costs [14]. To that end, non-metric topological representations have also been considered in classical literature [76], [77], and researchers have investigated the use of semantic topo maps for high-level navigation tasks [78], [79], [80]. Topo maps, based on low-dimensional graph representations, efficiently capture environment layouts and are beneficial for exploration or long-range planning.

In VLN, several works have employed topo maps and demonstrated superior performance [3], [16], [17], [54]. Because the long-range map can facilitate the agent to learn self-correction policy, which is crucial when the agent loses track of an instruction. However, these maps are derived from predefined graphs by marking observed nodes, which are unavailable in continuous or real-world environments. Chen et al. [18] explored topo maps in VLN-CE, but their proposed map is built offline through environment pre-exploration and assumes the agent has access to global topology priors, which limits its use in more realistic scenarios. Inspired by their novel ideas, we propose a more practical solution for topo mapping in VLN-CE. Without the need for predefined graphs or environment pre-exploration, our map is built online through the self-organization of predefined waypoints at each step. It is scalable as navigation progresses and meets the assumption of partial observability in a real environment.

III. METHOD

Task Setup: We address instruction-following navigation in indoor environments, where an agent is required to follow a specific path described by a natural language instruction to reach the target location. In particular, we focus on a practical setup - vision-language navigation in continuous environments (VLN-CE) [6], where the agent navigates on a 3D mesh of an environment with low-level actions. The action space consists of a set of parameterized discrete actions (e.g., FORWARD (0.25 m), ROTATE LEFT/RIGHT (15°), and STOP). VLN-CE uses the Habitat Simulator [61] to render environmental observations based on the Matterport3D scene dataset [81]. Following the panoramic VLN-CE setting [8], [9], [10], at each location, the agent receives panoramic RGB observations $O = \{I^{rgb}, I^d\}$ consisting of 12 RGB images and 12 depth images, which are captured from different views at 12 equally-spaced horizontal heading angles, i.e., $(0^\circ, 30^\circ, \dots, 330^\circ)$. The agent also receives an instruction for each episode. We denote the embeddings of the instruction with L words by $W = \{w_i\}_{i=1}^L$.

Overview of Our Approach: We propose a hierarchical navigation model, named 'ETPNav', which leverages high-level topological map-based planning and low-level controller for the VLN-CE task. As illustrated in Fig. 1, ETPNav comprises three modules: topological mapping, cross-modal planning and control. The mapping module maintains a topo map for each episode. Within each decision loop, the mapping module

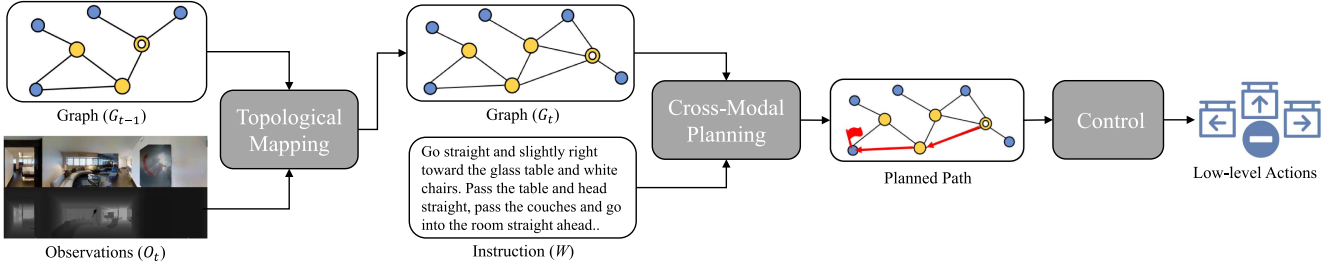


Fig. 1. Overview of the proposed model, ETPNav. It consists of three modules, a topological mapping module that gradually updates the topological map as it receives new observations, a cross-modal planning module that computes a navigational plan based on the instruction and map, and a control module that executes the plan with low-level actions.

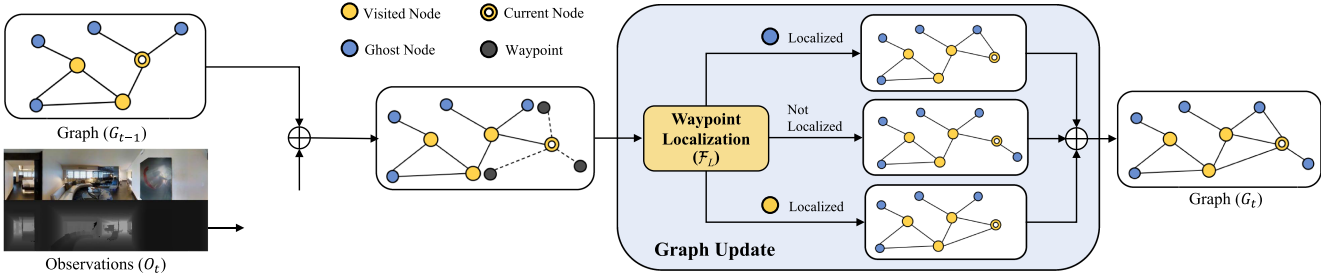


Fig. 2. Illustration of the topological mapping module. It takes the previous graph (G_{t-1}) and the agent observation (O_t) as input. The waypoint prediction submodule first predicts several nearby waypoints. The graph update submodule organizes these waypoints and incorporates them to update the graph using a waypoint localization function (F_L).

first updates the topo map by incorporate current observations. Subsequently, the planning module conducts cross-modal reasoning over the map and instruction to craft a high-level topological path plan. The plan is then executed by the control module, which drives the agent through a sequence of low-level actions. To be noted, in the following technique descriptions, we use ‘step t ’ to denote the decision loop step, rather than the low-level action step.

Similar to recent work [10], [14], [63], we presume that the agent can access the ground-truth pose provided by the simulator to facilitate mapping and control. Note that this work does not address the challenge of estimating pose based on noisy sensor readings. However, we suggest that visual odometry techniques [82] may be adaptable to our model in this context. This paper proceeds by introducing topological mapping in Section III-A, followed by cross-modal planning in Section III-B, and the presentation of our control policy in Section III-C. Finally, we provide detailed expositions of training and inference of our model in Section III-D.

A. Topological Mapping

To facilitate long-range planning, our agent constructs a topo map on the fly. This map shares a similar structure with [78], which abstracts the visited or observed locations along the traversed path as a graph representation, denoted as $G_t = \langle N_t, E_t \rangle$ at step t . Each node ($n_i \in N_t$) contains visual information observed at its location as well as position information. Two nodes are connected by an edge ($e_{i,j} \in E_t$) if their represented locations are directly reachable from each other. Each edge also

stores the relative euclidean distance between two nodes. We divide these nodes into visited nodes (yellow), the current node (yellow with a black dot), and ghost nodes (blue), where ‘ghost’ denotes that nodes have been observed but left unexplored.

Different from prior work [3], [16], [17], [18], our method assumes no prior knowledge of the environmental structure and we propose to construct the topo map via online self-organization of predicted waypoints. As depicted in Fig. 2, at each step t , the agent first predicts several nearby waypoints (black dots), representing possibly accessible locations near the agent. A current node (yellow with a black dot) is also initialized at the agent’s current location and connects to the last visited node (if it exists). The predicted waypoints and current node are represented by feature embeddings of the current observations O_t . These waypoints will be organized to update the previous topo map G_{t-1} and obtain the current map G_t .

Image Processing: Given the current step’s RGBD observations $O_t = \{I_t^{\text{rgb}}, I_t^{\text{d}}\}$, two different pre-trained visual encoders are used to extract RGB feature vectors $V_t^{\text{rgb}} = \{v_i^{\text{rgb}}\}_{i=1}^{12}$ and depth feature vectors $V_t^{\text{d}} = \{v_i^{\text{d}}\}_{i=1}^{12}$, respectively. To distinguish the features captured from different views of the panorama, we also apply orientation features $V_t^{\text{ori}} = \{(\cos \theta_i, \sin \theta_i)\}_{i=1}^{12}$, where θ_i represents heading angle. The parameters of the two visual encoders are fixed. More details of pre-processing are introduced in Section IV-A3.

Waypoint Prediction: We employ a transformer-based waypoint predictor [9] to generate the nearby waypoints. The predictor takes the depth feature vectors V_t^{d} and orientation feature vectors V_t^{ori} to predict the relative poses of these waypoints. Concretely, feature vectors in V_t^{d} and V_t^{ori} are first fused using a linear layer. The resulting vectors are fed into

a two-layer transformer to conduct inter-view interaction and obtain contextual depth embeddings. These embeddings are then fed into a multi-layer perceptron to obtain a heatmap representing probabilities of nearby waypoints in space. K waypoints $\Delta P^w = \{\Delta p_i^w\}_{i=1}^K$ are sampled from the heatmap using a non-maximum-suppression (NMS), where Δp_i^w denotes the relative pose to the agent. The predictor is pre-trained on the MP3D graph dataset [9], and its parameters are fixed.

To be noted, our predictor only takes depth images as inputs, instead of RGBD images used in [9]. Such depth-only design is motivated by the fact that waypoints only represent spatial accessibility, while semantic-level RGB information may be not helpful or even detrimental. We provide an ablation analysis of this design in Section IV-C1.

Visual Representations for Waypoints and the Current Node: We conduct feature mapping of the current observations O_t to represent the predicted waypoints and the current node. Specifically, RGB features V_t^{rgb} , depth features V_t^{d} and orientation features V_t^{ori} are fused using a linear layer, and then fed into a panorama encoder. The panorama encoder uses a multi-layer transformer to perform inter-view interaction and outputs contextual visual embeddings $\hat{V}_t = \{\hat{v}_i\}_{i=1}^{12}$. The current node $\odot$ has access to the panoramic observations and thus is represented as an average of $\hat{V}_t$. The waypoints $\bullet$ are partially observed and are represented by embeddings of views from which they can be observed. For example, if the relative heading angle of a waypoint to the agent is within $0^\circ \sim 30^\circ$, the waypoint is represented by the first view embedding $\hat{v}_1$. The waypoint representations will be incorporated to update the representations of ghost nodes $\bullet$.

Graph Update: We update the topo map with the predicted waypoints based on their spatial relations with existing nodes in the graph. This process utilizes a Waypoint Localization ($\mathcal{F}_L$) function to localize waypoints in the graph. $\mathcal{F}_L$ takes the position of a waypoint as input and computes its euclidean distances with all nodes in the graph. If the minimum distance is less than a threshold γ , $\mathcal{F}_L$ returns the corresponding node as the localized node. For each waypoint, we try to localize it in the graph using the Waypoint Localization function ($\mathcal{F}_L$). To update the graph, we divide the localization results into three cases:

- 1) If a visited node $\odot$ is localized, delete the input waypoint and add an edge between the current node and the localized visited node.
- 2) If a ghost node $\bullet$ is localized, accumulate the position and visual representation of the input waypoint to the localized ghost node. The new position and representation of the localized ghost node are updated as the average of its accumulated waypoint positions and representations.
- 3) If no node is localized, we take the input waypoint as a new ghost node.

B. Cross-Modal Planning

Fig. 3 illustrates the cross-modal planning module. It consists of a text encoder and a cross-modal graph encoder. The instruction of the current episode is encoded by the text encoder. Then, the cross-modal graph encoder conducts reasoning over

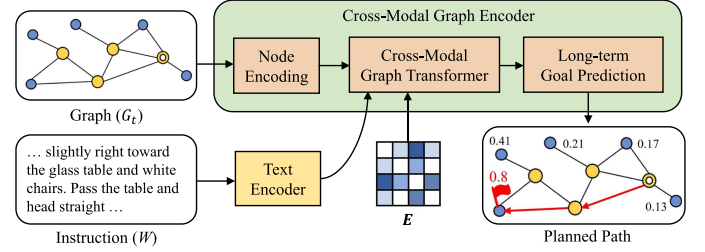


Fig. 3. Planning module consists of a text encoder for instruction encoding, and a graph encoder to conduct cross-modal reasoning over the map to generate a path plan.

the topo map and encoded instruction to predict a long-term goal node. The output is a planned topological path to the goal.

1) **Text Encoder:** Each word embedding w_i is added a positional embedding [83] corresponding to the position of the word in the sentence and a type embedding for text [84]. We denote the word embeddings with positional information as $\hat{W} = \{\hat{w}_i\}_{i=1}^L$. Those embeddings are then fed into a multi-layer transformer to obtain contextual word representations.

2) **Cross-Modal Graph Encoder:** The module takes the topo map G_t and encoded instruction $\hat{W}$ to predict a long-term goal node in the topo map.

Node Encoding: The visual feature in node n_i is added with a pose encoding and a navigation step encoding. The pose encoding embeds the global relative pose information of a node w.r.t. the agent's current location, including its orientation and euclidean distance relative to the current node. The navigation step encoding embeds the latest visited time step for visited nodes and 0 for ghost nodes. This allows visited nodes to be encoded with different histories to capture navigation dependencies and facilitate alignment with the instruction. The encoding of n_i is denoted as $\mathbf{n}_i$. To represent a STOP action, we add a 'stop' node in the graph and connect it with all other nodes.

Cross-Modal Graph Transformer: The encoded node and word embeddings are fed into a multi-layer transformer to conduct cross-modal interaction. The transformer architecture is similar to LXMERT [84], with each layer comprising one bi-directional cross-attention sub-layer, two self-attention sub-layers, and two feed-forward sub-layers. For node encoding, the standard self-attention layer [19] only considers visual similarity among nodes, which may overlook nearby nodes which are more relevant than distant nodes. To this end, we devise a graph-aware self-attention (GASA) that further takes into account the graph topology when computing inter-node attention for node encoding:

$$\text{GASA}(\mathbf{X}) = \text{softmax} \left(\frac{\mathbf{X} \mathbf{W}_q (\mathbf{X} \mathbf{W}_k)^\top}{\sqrt{d}} + \mathbf{E} \mathbf{W}_e \right) \mathbf{X} \mathbf{W}_v \quad (1)$$

where $\mathbf{X}$ represents the stack of all node encodings, $\mathbf{E}$ is the spatial matrix constructed by all-pair shortest distances obtained from the graph edges E_t , $\mathbf{W}_q, \mathbf{W}_k, \mathbf{W}_e, \mathbf{W}_v$ are learnable matrices. The produced visual-textual associated representation of nodes is formulated as $[\tilde{\mathbf{n}}_1, \dots, \tilde{\mathbf{n}}_{|N_t|}] = \text{GASA}([\mathbf{n}_1, \dots, \mathbf{n}_{|N_t|}])$.

Long-term Goal Prediction: We predict a navigation goal score for each node in the topo map G_t as follows:

$$s_i = \text{FFN}(\tilde{n}_i) \quad (2)$$

where FFN denotes a feed-forward network and $\tilde{n}_i$ is the multimodal representation of node n_i . Note that s_0 corresponds to the ‘stop’ node and it represents the score of the STOP action. To avoid unnecessary repeated visits to visited nodes, we mask the score for visited nodes and the current node. As such, a long-term goal is picked from ghost nodes or the ‘stop’ node.

Finally, the agent selects a long-term goal according to the predicted goal scores (e.g., pick the node with the maximum score). If the selected goal is the ‘stop’ node, navigation of the current episode terminates. If the selected goal is a ghost node, the agent computes a shortest path to the goal by performing Dijkstra’s algorithm on the graph. The resulting path plan consists of a sequence of subgoal nodes, denoted as $\mathcal{P}_t = \{p_m\}_{m=1}^M$ where p_m represents node position.

C. Control

The control module is responsible for converting the topological plan $\mathcal{P}_t$ into a series of low-level actions that guide the agent to the goal. Inputs for the control module include a sequence of subgoal nodes spanning $\mathcal{P}_t$, and the agent’s pose at each time step. The output action space of navigation control is a set of parameterized low-level actions defined by the VLN-CE task, e.g., FORWARD (0.25 m), ROTATE LEFT/RIGHT (15°), and STOP.

The control module produces actions that move the agent from one node to another. Similar to [8], [9], we employ a rotate-then-forward controller, named RF. Specifically, to reach a subgoal node p_m , RF first accesses the agent’s current pose and computes its relative orientation and distance ($\Delta\theta, \Delta\rho$) from p_m . Then, ($\Delta\theta, \Delta\rho$) are quantized and translated to a series of ROTATE (15°) actions, followed by a FORWARD (0.25 m) action sequence. RF executes these translated actions sequentially. After that, the current subgoal is consumed and the subsequent node in plan $\mathcal{P}_t$ becomes the new subgoal. The cycle repeats until no more nodes remain in $\mathcal{P}_t$.

Handling Unreachable Goal: It is possible that the predicted long-term goal (a ghost node) is unreachable, due to its position being estimated by predicted waypoints that might not be on the navigation mesh. In such cases, there is a risk of the agent repeatedly selecting the same unreachable goal node in alternating planning stages, inevitably leading to no progress in navigation control. To alleviate this issue, we employ a simple strategy - delete the selected ghost node from the graph map G_t before trying to reach it using navigation control. This approach not only avoids the repeated selection of unfeasible ghost nodes but also reduces the pool of candidates available for long-term goal prediction, thereby easing policy learning.

Obstacle Avoidance: The VLN-CE task simulates a practical navigation scenario where collision with obstacles is taken into account. Obstacle avoidance is essential, especially when sliding along obstacles is forbidden, such as on the RxR-CE dataset [22]. In such cases, the agent is unable to move forward if its chassis

comes into contact with an obstacle. This can result in navigation deadlocks, and in extreme cases, navigation failure due to early episode termination. To address this issue, we devise a trial-and-error heuristic called ‘Tryout’ to prevent navigation deadlocks. ‘Tryout’ comes into play when the RF controller executes the FORWARD flow, and it shares a similar spirit with the ‘brute force untrap mode’ [85]. Concretely, it detects deadlocks by checking if the agent’s position changes after executing a FORWARD action. If a deadlock is identified, the Tryout compels the agent to rotate with a set of predefined headings $\Delta\Theta^{\text{try}}$ and attempt to move on with a single FORWARD action. If the agent moves away from its previous position, it indicates that the agent has exited the dead-end. Then, the agent returns to its original heading and continues with the remaining FORWARD control flow. However, if the agent’s position remains unchanged, it proceeds to try other headings in $\Delta\Theta^{\text{try}}$. In practice, $\Delta\Theta^{\text{try}}$ consists of 7 equally-space horizontal heading angles, ranging from 90° counterclockwise (−90°) to 90° clockwise (90°).

D. Training and Inference

Pre-training: We pre-train the planning module with proxy tasks to improve its generalization ability, following the common practice in transformer-based VLN models [4], [5], [49]. In this stage, the input topo maps are constructed offline and derived from the predefined graphs used in Matterport3D simulator [1]. Specifically, given an expert trajectory, we first extract its corresponding sub-graph from the predefined graph, and then mark the current node, visited nodes, and ghost nodes along the trajectory. Further, we align rendered RGBD images in the Habitat Simulator [61] onto the predefined graph for feature mapping in the map construction process. We adopt Masked Language Modeling (MLM) [83] and Single Action Prediction (SAP) [47] proxy tasks for pre-training. In the MLM task, the input instructions are randomly masked and the planning module is optimized by recovering the masked words after map-instruction interaction as described in Section III-B. As for the SAP task, we randomly chunk an input expert trajectory and build its corresponding topo map. The objective of this task is to predict the next teacher action, i.e., the subsequent action node of the chunked trajectory.

Fine-tuning: We then fine-tune our model on downstream VLN-CE tasks to adapt navigation on 3D meshes in the Habitat Simulator [61]. To avoid overfitting to expert experience, we use ‘student-forcing’ [6] to train the model, where the predicted long-term goal of each step is sampled through the probability distribution of the predicted scores (2). In each decision loop, the agent updates the topo map as described in Section III-A, and then conducts cross-modal map-instruction reasoning to predict a long-term goal as explained in Section III-B. The planned path is executed by a controller as presented in Section III-C. Similar to DAgger [86], we employ an interactive demonstrator * to determine the teacher action node of each step. The demonstrator * accesses the ground-truth 3D mesh and outputs one ghost node as the teacher supervision. * takes different strategies on different datasets. On the R2R-CE dataset, the teacher action node a_t^* is the ghost node that has the shortest geodesic distance

TABLE I
DATA STATISTICS AND AGENT EMBODIMENT OF R2R-CE AND RxR-CE DATASETS

Dataset	Language	Length		Train		Val-Seen		Val-Unseen		Test-Unseen		Embodiment	
		Path	Sentence	#house	#instr	#house	#instr	#house	#instr	#house	#instr	Chassis	Sliding
R2R-CE	en	9.89m	32 words	61	10,819	53	778	11	1,839	18	3,408	0.10m	Allowed
RxR-CE	en, hi, te	15.23m	120 words	59	60,300	57	6,746	11	11,006	17	9,557	0.18m	Forbidden

to the final target. On the RxR-CE dataset that does not has shortest-path prior, * takes a path-fidelity strategy similar to [7]. Concretely, the annotated reference path is discretized as a sequence of subgoals. * keeps track of visited subgoals and a_t^* is the ghost node that has the shortest geodesic distance to the next unvisited subgoal. Overall, the policy learning objective is formulated as:

$$L = \sum_{t=1}^T -\log p(a_t^* | W, G_t) \quad (3)$$

Inference: During the testing phase, the agent consistently runs the mapping-planning-control cycle, which is analogous to the fine-tuning stage. The primary distinction between the two stages pertains to the long-term goal sampling strategy employed at each planning step. In this case, the agent greedily selects the ghost node with the maximum predicted scores (2). In the event that the agent triggers a STOP action or surpasses the maximum number of goal predictions, the ongoing episode's navigation will terminate. In line with [9], the maximum number of predictions is set as 15 for the R2R-CE dataset and 25 for the RxR-CE dataset.

IV. EXPERIMENT

A. Experimental Setup

1) *Datasets:* We conduct experiments on R2R-CE and RxR-CE datasets, which are created by converting discrete paths of R2R [1] and RxR [22] datasets into continuous environments through the Habitat Simulator [61]. While both datasets provide step-by-step language guidance, they differ in various aspects such as path length, guidance granularity, and agent embodiment as summarized in Table I.

The R2R-CE dataset comprises a total of 5,611 shortest-path trajectories, encompassing train, validation, and test splits. Each trajectory corresponds to approximately 3 English instructions. The average path length is 9.89 m and each instruction consists of an average of 32 words. We report performance on several validation splits. Val-Seen contains episodes with novel paths and instructions but from scenes observed in training. Val-Unseen contains novel paths, instructions, and scenes. Agents in R2R-CE have a chassis radius of 0.10 m and can slide along obstacles while navigating.

RxR-CE is larger and more challenging compared to R2R-CE. While having similar scene splits as R2R-CE, RxR-CE presents substantively more instructions, spanning multilingual descriptions in English, Hindi, and Telugu, requiring an average of 120 words per instruction. Additionally, annotated paths in RxR-CE are much longer than those in R2R-CE (15.23 m versus 9.89 m). To be noted, agents in RxR-CE are forbidden to slide

along obstacles, and the larger chassis radius (0.18 m) makes it prone to collide with obstacles. This also makes RxR-CE more challenging because navigation can easily get stuck when encountering obstacles, underscoring the vital role of obstacle avoidance in this challenging task.

2) *Evaluation Metrics:* Following previous works [1], [87], [88], we adopt the following navigation metrics. Trajectory Length (TL): average path length in meters; Navigation Error (NE): average geometric distance in meters between the final and target location; Success Rate (SR): the ratio of paths with NE less than 3 meters; Oracle SR (OSR): SR given oracle stop policy; SR penalized by Path Length (SPL); Normalize Dynamic Time Wrapping (NDTW): the fidelity between the predicted and annotated paths and NDTW penalized by SR (SDTW). R2R-CE uses SR and SPL as its primary metrics, whereas RxR-CE is more concerned with path fidelity and uses NDTW and SDTW as its primary metrics.

3) *Implementation Details. Model Configuration:* For visual encoding, we use ViT-B/32 [89] pre-trained in CLIP [90] to encode RGB images as [11], and ResNet-50 [91] pre-trained in point-goal navigation [13] to encode depth images following [6]. The same as [5], [9], [84], we set the layers' number of the panorama encoder, the text encoder, and the cross-modal graph encoder as 2, 9, 4, respectively. Other hyperparameters are the same as LXMERT [84] (e.g., the hidden layer size is 768). In the pre-training stage, we initialize the model with pre-trained LXMERT on the R2R-CE dataset and pre-trained RoBERTa [92] for the multilingual RxR-CE dataset.

Training Details: Our experiments were performed using the PyTorch framework [93] and executed on two NVIDIA RTX 3090 GPUs. Our model includes two trainable modules: the panorama encoder used in the topological mapping module, and the cross-modal planning module. We pre-train our model for 100,000 iterations (~ 20 hours), with a batch size of 64 and a learning rate of $5e-5$, utilizing the AdamW optimizer [94]. In this stage, topological maps are built offline and derived from the pre-defined graph of discrete VLN [1]. We leverage the discrete paths in the R2R and RxR datasets for pre-training purposes and augment the data using synthetic instructions from Prevalent [47] and RxR-Markey [43]. After pre-training, we choose the model weights, producing the best zero-shot navigation performance (e.g., SPL on R2R-CE, SDTW on RxR-CE) to initialize the fine-tuning stage. During fine-tuning, the agent interacts with the environments online through the Habitat Simulator [61] and is supervised by the teacher node generated by the demonstrator *. We leverage scheduled sampling [95] to train the model, shifting from teacher-forcing to student-forcing with a decay frequency of per 3000 iterations and decay ratio of 0.75. The fine-tuning iterations amount to 15,000 (~ 30 hours) with a batch size of 16

TABLE II
COMPARISON WITH STATE-OF-THE-ART METHODS ON R2R-CE DATASET

Methods	Val Seen					Val Unseen					Test Unseen				
	TL	NE↓	OSR↑	SR↑	SPL↑	TL	NE↓	OSR↑	SR↑	SPL↑	TL	NE↓	OSR↑	SR↑	SPL↑
Seq2Seq [6]	9.26	7.12	46	37	35	8.64	7.37	40	32	30	8.85	7.91	36	28	25
SASRA [62]	8.89	7.71	-	36	34	7.89	8.32	-	24	22	-	-	-	-	-
CMTP [18]	-	7.10	56	36	31	-	7.90	38	26	23	-	-	-	-	-
LAW [7]	-	-	-	40	37	-	-	-	35	31	-	-	-	-	-
HPN [8]	8.54	5.48	53	46	43	7.62	6.31	40	36	34	8.02	6.65	37	32	30
CM2 [14]	12.05	6.10	51	43	35	11.54	7.02	42	34	28	13.90	7.70	39	31	24
WS-MGMAP [63]	10.12	5.65	52	47	43	10.00	6.28	48	39	34	12.30	7.11	45	35	28
CWP-CMA [9]	11.47	5.20	61	51	45	10.90	6.20	52	41	36	11.85	6.30	49	38	33
CWP-RecBERT [9]	12.50	5.02	59	50	44	12.23	5.74	53	44	39	13.51	5.89	51	42	36
Sim2Sim [10]	11.18	4.67	61	52	44	10.69	6.07	52	43	36	11.43	6.17	52	44	37
Reborn (ours) [11]	10.29	4.34	67	59	56	10.06	5.40	57	50	46	11.47	5.55	57	49	45
ETPNav (ours)	11.78	3.95	72	66	59	11.99	4.71	65	57	49	12.87	5.12	63	55	48

TABLE III
COMPARISON AGAINST OTHER VLN PLANNERS

Planners	R2R-CE Val-Unseen				
	TL	NE↓	OSR↑	SR↑	SPL↑
RecBERT [9]	12.46	5.66	54.13	45.14	39.87
HAMT [5]	10.23	5.41	57.09	49.97	45.72
DUET [17]	13.08	5.16	62.21	53.71	46.43
ETPNav (Ours)	11.99	4.71	64.71	57.21	49.15

and a learning rate of $1e-5$. The best iterations are determined by best performance on validation unseen splits.

B. Comparison With State-of-the-Art Methods

1) *R2R-CE*: In Table II, we compared our ETPNav with current state-of-the-art methods on the R2R-CE dataset. The results demonstrate that our model outperforms the existing models on all splits in terms of NE, OSR, SR, and SPL. Particularly, on the val unseen split, ETPNav surpasses the second-best model CWP-RecBERT [9] by 13 SR and 10 SPL. Moreover, our model also generalizes well on the test unseen split, as it outperforms Sim2Sim [10] by 11 SR and 11 SPL. Reborn [11] serves as the initial version for the 2022 RxR-Habitat Challenge. It uses a local planning space, which consists of nearby waypoints, and utilizes an unstructured memory bank to capture navigation dependency. The performance gap between Reborn and ETPNav is substantial, with ETPNav outperforming Reborn on the test unseen split by 6 SR and 3 SPL. This highlights the efficacy of global planning with topo maps, enabling the agent to encode structured environmental priors and allowing for long-term planning, leading to a more robust policy. We also compare ETPNav against several advanced VLN planners in Table III. Specifically, RecBERT [9], HAMT [5] and DUET [17] are transferred into VLN-CE where we align their controllers and model configurations with ETPNav's (e.g., the CLIP-ViT-B/32 [90] visual encoder, depth-only waypoint and topological map). ETPNav consistently surpasses these alternatives by a clear margin (e.g., 3.5 SR and 2.72 SPL higher than DUET). This further demonstrates the efficacy of ETPNav's planning capability.

2) *RxR-CE*: Table IV compares our ETPNav model with the current state-of-the-art methods on the RxR-CE dataset. Our model outperforms the existing best model, CWP-RecBERT [9] on all evaluation metrics on the three splits. For instance, on the val unseen split, ETPNav surpasses CWP-RecBERT by 27.71 SR, 22.24 SPL, and 15.19 NDTW. ETPNav also generalizes well on the test unseen split, where it outperforms CWP-RecBERT by 26.36 SR, 20.25 SPL, 16.81 NDTW, and 22.25 SDTW. For a fair comparison, we also report our results without Marky-mT5 [43] data augmentation, where ETPNav still beats CWP-RecBERT by a significant margin, for instance, $\uparrow 25.99$ SR and $\uparrow 14.78$ NDTW on the val unseen split. Please note that Reborn [11] is our winning entry for the 2022 RxR-Habitat Challenge, which employs a local planning space composed of nearby waypoints. While Reborn achieves slightly better NDTW (e.g., 55.43 versus 54.11) on the test unseen splits, it has significant worse SDTW (e.g., 38.43 versus 41.30). We attribute this to the global planning space of ETPNav, which promotes backtracking and may impact path fidelity. However, this global planning space enables the agent to make long-term plans, resulting in better SR and SDTW.

C. Ablation Study

In this section, we provide detailed ablation experiments to evaluate specific components of ETPNav, including critical design choices of the topological mapping module (Section IV-C1) and the cross-modal planning module (Section IV-C2). Additionally, we compare the proposed controller with other alternatives (Section IV-C3). Finally, we visualize the trajectories predicted by our model and compare them with other variants (Section IV-C4).

1) *Key Design Choices of Topological Mapping. Waypoint Prediction*: Table V presents a comparison between three different waypoint predictors on the R2R-CE dataset. In Row 1, RGB and depth features are utilized as inputs, where both feature types are linearly transformed to the same dimension, fused, and then fed into the transformer layer to predict waypoints. This approach is also the default choice in [9]. Row 2 only takes RGB features as inputs, while Row 3 shows our approach that uses only depth features for waypoint prediction. We apply

TABLE IV
COMPARISON WITH STATE-OF-THE-ART METHODS ON RXR-CE DATASET

Methods	Val Seen					Val Unseen					Test Unseen				
	NE↓	SR↑	SPL↑	NDTW↑	SDTW↑	NE↓	SR↑	SPL↑	NDTW↑	SDTW↑	NE↓	SR↑	SPL↑	NDTW↑	SDTW↑
Seq2Seq [6]	-	-	-	-	-	-	-	-	-	-	12.10	13.93	11.96	30.86	11.01
CWP-CMA [9]	-	-	-	-	-	8.76	26.59	22.16	47.05	-	10.40	24.08	19.07	37.39	18.65
CWP-RecBERT [9]	-	-	-	-	-	8.98	27.08	22.65	46.71	-	10.40	24.85	19.61	37.30	19.05
Reborn†(ours) [11]	5.73	51.14	44.78	65.72	43.84	5.82	47.56	41.65	63.02	41.16	-	-	-	-	-
Reborn (ours) [11]	5.69	52.43	45.46	66.27	44.47	5.98	48.60	42.05	63.35	41.82	7.10	45.82	38.82	55.43	38.42
ETPNav†(ours)	5.55	57.26	47.67	64.15	47.57	5.80	53.07	44.16	61.49	43.92	-	-	-	-	-
ETPNav (ours)	5.03	61.46	50.83	66.41	51.28	5.64	54.79	44.89	61.90	45.33	6.99	51.21	39.86	54.11	41.30

†Results without Marky-mT5 synthetic instructions [43].

TABLE V
COMPARISON OF DIFFERENT WAYPOINT PREDICTORS

#	inputs	Waypoint Prediction				R2R-CE Val-Seen					R2R-CE Val-Unseen				
		$ \Delta $	%Open↑	d_C ↓	d_H ↓	TL	NE↓	OSR↑	SR↑	SPL↑	TL	NE↓	OSR↑	SR↑	SPL↑
1	RGBD	1.40	82.87	1.05	2.01	11.22	3.87	70.56	63.88	57.11	11.77	4.73	63.24	56.44	48.53
2	RGB	1.38	65.34	1.08	2.03	13.38	4.38	63.49	56.29	46.42	12.81	4.99	57.91	51.66	42.21
3	Depth	1.39	84.05	1.04	2.01	11.78	3.95	71.85	66.19	59.37	11.99	4.71	64.71	57.21	49.15

waypoint metrics [9] and navigation results to assess the quality of predicted waypoints. These waypoint metrics are as follows: $|\Delta|$ measures the difference in the number of target waypoints and predicted waypoints. %Open measures the ratio of waypoints that is in open space (not hindered by any obstacle). d_C and d_H are the Chamfer distance and the Hausdorff distance, respectively, commonly used metrics to measure the distance between point clouds.

As shown in Table V, Row 1 achieves a decent performance in both waypoints and navigation metrics on the val unseen split, with 82.87 %Open, 1.05 D_C , and 56.44 SR. Conversely, Row 2 only utilizes RGB to predict waypoints, resulting in the worst performance of all with 65.34 %Open and 1.08 D_C . Without depth information, the %Open metric drops severely, indicating that many waypoints are obstructed by obstacles or not on the navigation mesh. Consequently, the navigation performance also declines considerably, for example, compared to Row 1, SR drops by 4.78 on the val unseen split. It is noteworthy that the depth-only predictor (Row 3) yields the best performance, achieving 84.05 %Open and 1.04 D_C . Additionally, the navigation performance is also superior, with 57.21 SR and 49.15 SPL on the val unseen split, compared to 56.44 SR and 48.53 SPL respectively. These findings suggest that RGB information is ineffective and even detrimental to waypoint prediction. One possible explanation is that low-level semantics in RGB features can make the predictor overfit to seen environments, while such semantics are unnecessary for inferring spatial accessibility.

Different Options for Map Construction: Table VI compares different options for map construction on the R2R-CE dataset, including the localization threshold γ and the waypoint accumulation in Section III-A, as well as the ghost node deleting in Section III-C.

As the localization threshold γ increases, the number of nodes N_{node} shows a downward trend. This is because a higher γ encourages the agent to localize predicted waypoints onto existing nodes of the graph, thereby reducing the creation of

TABLE VI
COMPARISON OF DIFFERENT OPTIONS FOR MAP CONSTRUCTION

#	γ (m)	Acc.	Del.	R2R-CE Val-Unseen					
				N_{node}	TL	NE↓	OSR↑	SR↑	SPL↑
1	0.25	✓	✓	32.18	11.68	4.70	63.34	56.71	48.71
2			✓	34.13	11.33	4.81	62.26	55.19	48.30
3		✓		30.80	13.46	5.08	62.15	53.34	45.64
4	0.50	✓	✓	23.76	11.99	4.71	64.71	57.21	49.15
5			✓	31.02	12.61	4.68	63.13	55.89	47.92
6		✓		21.01	13.38	5.03	59.70	52.41	45.02
7	0.75	✓	✓	18.23	13.97	4.94	64.11	54.75	45.42
8			✓	25.45	14.48	4.81	61.06	53.61	45.86
9		✓		15.57	15.04	5.07	58.78	51.16	42.38
10	1.00	✓	✓	14.43	18.60	6.13	52.31	42.30	33.60
11			✓	20.29	21.02	5.68	53.12	41.59	32.17
12		✓		10.98	18.52	5.50	49.32	42.36	33.09

γ is the threshold of the waypoint localization function F_L . ‘Acc.’ denotes accumulating multiple waypoints to represent one ghost node. ‘Del.’ denotes deleting the selected ghost node in each planning step. N_{node} denotes the average number of nodes per episode.

new nodes. Meanwhile, the overall navigation performance is sensitive to the number of nodes N_{node} . For example, on the val unseen split, there is approximately 12 SR drop comparing (Row 10 ~ Row 12) to (Row 1 ~ Row 3). The reason is that a higher γ results in too few nodes to depict the environment well, limiting the agent’s accurate perception and efficient planning. However, a large N_{node} also affects the navigation performance, e.g., on the val unseen split, 56.71 SR of Row 1 versus 57.21 SR of Row 4. One potential reason is that a larger number of candidate nodes increases the learning difficulty of the planning module.

Moreover, both the ‘Acc.’ and ‘Del.’ are beneficial. For instance, comparing Row 4 and Row 5, without ‘Acc.’, SR and SPL on the val unseen split decrease by 1.32 and 1.23 respectively. ‘Acc.’ allows the agent to integrate multi-step waypoint observations to represent ghost nodes, promoting the planning

TABLE VII
COMPARISON OF DIFFERENT PLANNING SPACES (LOCAL VERSUS GLOBAL)

#	Planning Space	GASA	R2R-CE Val-Unseen				
			TL	NE↓	OSR↑	SR↑	SPL↑
1	Local	✓	11.37	4.92	61.28	53.15	46.83
2			12.12	4.94	62.18	53.92	46.43
3	Global	✓	12.04	4.83	63.48	55.97	48.08
4			11.99	4.71	64.71	57.21	49.15

GASA represents graph-aware self-attention.

TABLE VIII
EFFECT OF PRE-TRAINING TASKS

#	Proxy Tasks	R2R-CE Val-Unseen				
		TL	NE↓	OSR↑	SR↑	SPL↑
1	No init	14.35	6.81	49.21	37.41	30.28
2	MLM	16.69	5.44	58.51	48.23	37.73
3	SAP	13.05	5.04	59.98	52.37	42.77
4	MLM + SAP	11.99	4.71	64.71	57.21	49.15

module to predict an accurate long-term goal. Similarly, comparing Row 4 and Row 6, without ‘Del.’, the performance decreases significantly, with SR and SPL on the val unseen split dropping by 4.80 and 4.13 respectively. Without ‘Del.’, unreachable ghost nodes can be selected endlessly by the agent, resulting in no progress in navigation. In subsequent experiments, Row 4 is taken as the default setup.

2) *Key Design Choices of Cross-Modal Planning. Comparison of Different Planning Space:* Table VII compares different planning spaces on the R2R-CE dataset as well as the effect of GASA in (1). The local planning space only considers adjacent ghost nodes of the agent as candidate goals, while the global planning space consists of all observed ghost nodes along the traversed path. Global planning results in better navigation performance, e.g., on the val unseen split, Row 4 achieves a 57.21 SR compared to Row 2 at 53.92 SR. This demonstrates the superiority of global planning, as it allows efficient backtracking to a previous location, providing self-correction policy. In contrast, local planning requires multiple plan-control flows to reach a remote location, introducing unstable accumulation bias, making it challenging to achieve such intelligent behavior. GASA is also shown to be effective as it increases SR by about 1 point compared (Row 2, Row 4) to (Row 1, Row 3) where it is not used. GASA introduces topology for node encoding, facilitating the agent’s ability to capture environmental structural priors. We also note that the gain of GASA to global planning is more significant than that to local planning, comparing $\uparrow 1.24$ SR in global planning and $\uparrow 0.77$ SR in local planning. We suspect this because conducting global planning requires an understanding of the house structure, while local planning is restricted to nearby areas, thereby reducing the need for structure priors.

The Effect of Pre-training: Table VIII presents the benefits of various pre-training tasks on the downstream R2R-CE task. Row 1 shows results from the model trained from scratch, which shows the worst performance (e.g., on the val unseen split, 37.41 SR and 30.28 SPL). Row 2 displays the outcomes of

TABLE IX
COMPARISON OF DIFFERENT VISUAL INPUTS IN PRE-TRAINING

#	RGB	Depth	R2R-CE Val-Unseen				
			TL	NE↓	OSR↑	SR↑	SPL↑
1	MP3D	✓	13.54	5.04	62.04	53.24	44.09
2	MP3D		13.13	4.98	64.49	55.74	47.48
3	Habitat	✓	12.90	4.98	62.59	55.92	46.97
4	Habitat		11.99	4.71	64.71	57.21	49.15

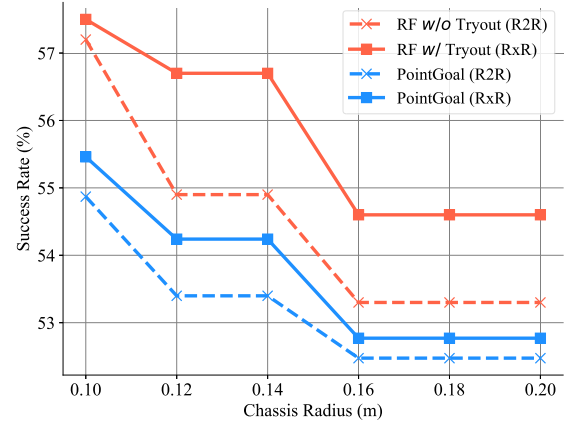


Fig. 4. Effect of the agent’s chassis radius on SR.

pre-training with the MLM task, indicating a significant gain from the generic pre-training task (e.g., $\uparrow 10.82$ SR and $\uparrow 7.45$ SPL on the val unseen split). Because the MLM task enables the model to learn transferable visiolinguistic representations, enhancing the agent’s generalization ability. Row 3 shows the efficacy of the SAP task. Compared to row 1, the navigation performance increases significantly with $\uparrow 14.96$ SR and $\uparrow 12.49$ SPL. Because SAP can promote the model to learn navigation-oriented representations, which is crucial for navigation policy learning. In row 4, the agent achieves the best performance when combining MLM and SAP tasks, demonstrating the two proxy tasks are complementary. Thus, we assemble MLM and SAP as our default pre-training tasks.

Table IX presents the effect of using different visual inputs for pre-training on the downstream R2R-CE task. Row 1 and Row 2 pre-train the model with RGB images captured in Matterport3D Simulator [1], which is a common practice in existing pre-training-based VLN-CE models [9], [10], [11]. In contrast, Row 3 and Row 4 use RGB images reconstructed in Habitat Simulator [61]. Notably, Row 3 outperforms Row 1 by 2.68 SR and 2.88 SPL on the val unseen split, highlighting a performance gap between MP3D and Habitat simulators due to their visual domain gap. Although the model can be fine-tuned using habitat images, the performance loss caused by the domain gap in the pre-training stage cannot be eliminated. Additionally, adding depth information for pre-training (e.g., Row 4) shows better performance than Row 3, with an improvement of 1.29 SR and 2.18 SPL on the val unseen split. Therefore, Row 4 is used as our default pre-training setup.

TABLE X
COMPARISON OF DIFFERENT CONTROLLERS

#	Controllers	R2R-CE Val-Unseen					RxR-CE Val-Unseen				
		TL	NE↓	OSR↑	SR↑	SPL↑	NE↓	SR↑	SPL↑	NDTW↑	SDTW↑
1	Teleportation	11.31	4.64	65.14	57.97	49.76	5.80	54.98	45.23	64.33	46.04
2	PointGoal [13]	13.35	4.87	63.40	54.86	44.05	5.89	52.77	43.50	61.03	43.79
3	FMM [12]	12.62	5.19	61.45	52.64	44.34	8.83	25.95	23.02	47.29	21.73
4	FMM <i>w/</i> CMap	12.93	5.12	62.43	53.62	44.96	6.02	50.06	40.07	58.75	41.09
5	RF <i>w/o</i> Tryout	11.99	4.71	64.71	57.21	49.15	9.22	22.61	20.06	44.30	18.64
6	RF <i>w/</i> Tryout	-	-	-	-	-	5.64	54.79	44.89	61.90	45.33

TABLE XI
QUANTITATIVE RESULTS OF THE ORACLE PLANNER WITH DIFFERENT CONTROLLERS

#	Controllers	R2R-CE Val-Unseen							RxR-CE Val-Unseen						
		TL	AT	RT	SG-NE↓	CT↓	SR↑	SPL↑	TL	AT	RT	SG-NE↓	CT↓	SR↑	NDTW↑
1	PointGoal [13]	10.58	146.81	33.99	0.17	6.62	94.77	91.14	16.58	223.16	69.12	0.31	45.23	83.37	79.32
2	FMM [12]	10.13	188.79	52.12	0.16	6.37	94.34	87.68	7.99 *	385.23	46.23	0.50	231.62	41.40	59.55
3	FMM <i>w/</i> CMap	10.20	188.44	52.64	0.15	4.59	94.45	87.68	16.75	281.50	69.34	0.33	40.36	77.57	77.09
4	RF <i>w/o</i> Tryout	9.56	60.63	21.18	0.15	1.78	97.61	93.27	7.49 *	148.59	49.79	0.83	70.04	35.92	54.61
5	RF <i>w/</i> Tryout	-	-	-	-	-	-	-	15.46	148.58	64.80	0.28	17.22	88.85	82.34

* Much lower TL due to frequent collisions and deadlocks.



Fig. 5. Comparison of the same episode's trajectories predicted by different model variants. (Top) The trajectory predicted by ETPNav using local planning. (Bottom) The trajectory predicted by ETPNav using global planning. .

3) *Comparison of Different Controllers:* Table X compares the performance of various navigation controllers on the R2R-CE and RxR-CE datasets. The Teleportation controller serves as the performance upper bound. It transports the agent to the goal predicted by the planning module. Since the goal (a ghost node) might not be on the navigation mesh, in practice, we first transport the agent to an adjacent node of the goal, then drive it towards the goal using the proposed controller.

Other alternative controllers include PointGoal, Fast Marching Method and RF. PointGoal [13] employs a pre-trained LSTM to predict low-level actions to reach the goal localization. We follow [14] to adapt PointGoal to the VLN-CE task. Fast Marching Method (FMM) [12] is a classical obstacle-aware controller which constructs a local occupancy map on the fly for path planning. This map is aggregated by the geometric projection of depth observations. We follow [10] to configure

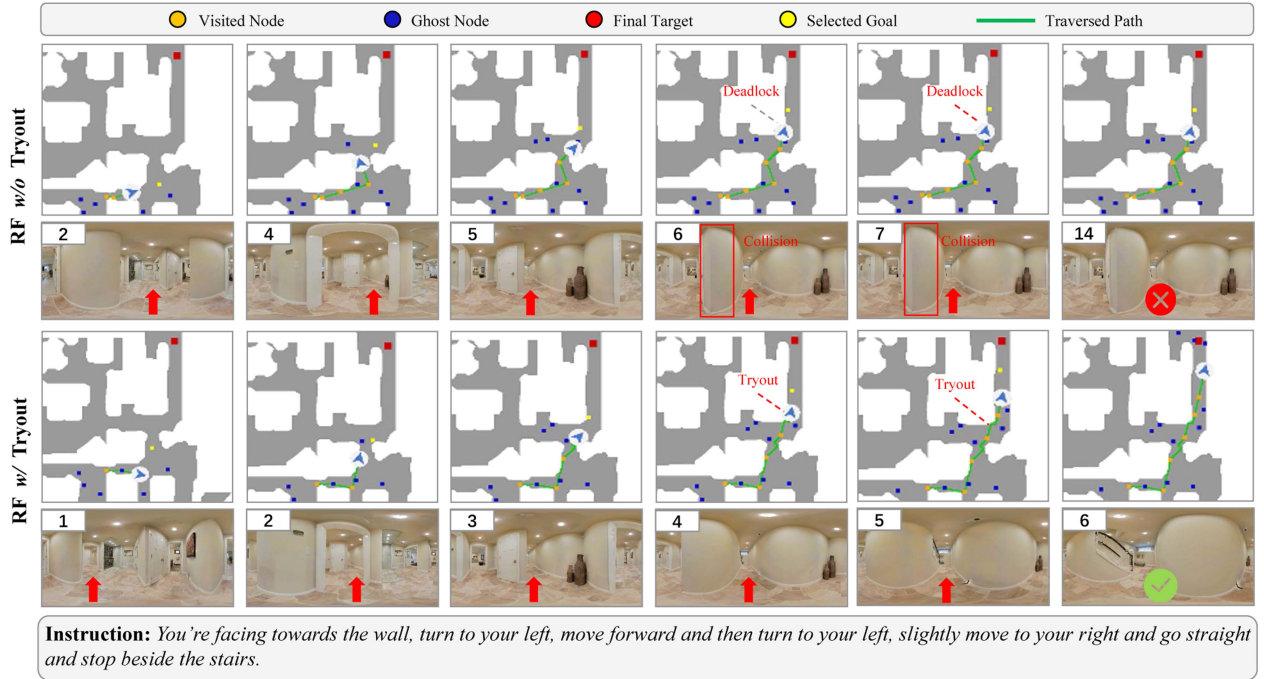


Fig. 6. Comparison of the same episode's trajectories predicted by different model variants. (Top) The trajectory predicted by ETPNav without Tryout control. (Bottom) The trajectory predicted by ETPNav with Tryout control. .

FMM on the VLN-CE task. We further consider an FMM variant (i.e., FMM w/ CMap) to handle invisible obstacles due to simulation error, where obstacle collisions are recorded as a collision map (CMap) similar to [85]. The CMap is then combined with the local occupancy map for path planning. RF is the proposed controller described in Section III-C and Tryout is active when sliding along obstacles is forbidden (i.e., RxR-CE dataset).

Row 1 establishes the upper bound, reaching 57.97 SR and 49.76 SPL on the R2R-CE dataset and 64.43 NDTW and 46.04 SDTW on the RxR-CE dataset. In Row 2, PointGoal shows satisfactory performance, but there is a clear gap to Row 1 with a decrease of 5.71 SPL on the R2R-CE dataset and 2.25 SDTW on the RxR-CE dataset. Row 3 and Row 4 show FMM could achieve decent performance on the R2R-CE dataset, whereas the collision map (Row 4) yields marginally superior outcomes. Moreover, the collision map shows to be critical on the sliding-forbidden RxR-CE dataset, where the performance drops severely without it (e.g., Row 3's 21.73 SDTW versus Row 4's 41.09). Because the occupancy map alone is not enough for obstacle-avoiding, the agent in Row 3 gets stuck frequently. More analyses will be presented in the next paragraph. Row 5 shows that our RF controller manages to narrow the gap on the R2R-CE dataset, reaching 49.15 SPL compared to Row 1's 49.76 SPL. However, RF results in significant performance drops on the RxR-CE dataset with a 27.4 SDTW decrease compared to Row 1. Because it is unaware of collision and causes frequent deadlocks in obstacles under the challenging sliding-forbidden setup, resulting in navigation failure. The proposed Tryout satisfactorily handles this problem, nearly eliminating the performance loss caused by sliding-forbidden with 45.33 SDTW on Row 6 compared to 46.04 SDTW on Row 1. Tryout even

surpasses the learning-based PointGoal controller with 45.33 SDTW on Row 6 compared to 43.79 SDTW on Row 2.

In Table XI, we evaluate different controllers with an oracle planner, which always selects the teacher action node at each planning step. Four extra metrics are considered, namely Actions Taken (AT), Rotations Taken (RT), Subgoal Navigation Error (SG-NE), and Collision Times (CT). AT and RT are the average numbers of actions and rotations (ROTATE LEFT/RIGHT) taken per episode. SG-NE is the average distance between the teacher action node and the agent's position after control execution. CT denotes the average times of obstacle collisions per episode. We summarize our findings below. First, compared to other controllers, RF variants (Row 4 and Row 5) take fewer actions and have shorter paths, which can contribute to their higher navigation efficiency (SPL) and path fidelity (NDTW). This is because the rotate-then-forward control flow forces the agent to walk in a straight line, while others may take unnecessary rotations to keep the agent away from obstacles. Second, though RF controllers simply walk in a straight line, they do not lead to significantly more collisions. We attribute this to the dataset [9] used to train our waypoint predictor, which makes the waypoints typically lie in straight-line accessible open spaces. This can also explain why Tryout can avoid deadlocks with a few attempts (Row 4 versus Row 5). Third, learning-based PointGoal can avoid obstacles implicitly (Row 1), while classical FMM failed to avoid obstacles and resulting in the highest CT (Row 2's 231.62 CT on the RxR-CE dataset). This indicates obstacle-avoiding relying on the occupancy map alone is difficult and a similar phenomenon is also observed in [85]. This is due to the imperfect occupancy map caused by obstacles out of view, inaccurate depth observations, and artifacts in scans of indoor

spaces [6]. Row 3 mitigates this issue with collision information, and our Tryout also has a similar spirit.

Also, we are interested in how the agent's chassis radius may impact navigational performance. Fig. 4 shows the episodic success rate of ETPNav on the R2R-CE and RxR-CE datasets whether employing PointGoal or RF controllers. For all datasets, the success rate of the two controllers decreases as the chassis radius increases since a bigger chassis might lead the agent to collide with obstacles more frequently, increasing the risk of navigation failure. The proposed RF controller, however, is resilient to this problem and consistently outperforms the PointGoal controller, particularly on the RxR-CE dataset, where it beats PointGoal by about 3 SR through all chassis radiuses. The main reason is that it uses Tryout to explicitly prevent the agent from getting stuck in obstacles, which aids the adaptation of navigation policy to various chassis sizes. This further verifies the robustness of the proposed obstacle-avoiding controller.

4) *Qualitative Results:* Figs. 5 and 6 visualize trajectories predicted by our model compared to the variant using local planning (on the R2R-CE dataset) and the variant without Tryout control (on the RxR-CE dataset), respectively.

As shown in Fig. 5, the local planning space is insufficient to capture the global environment layouts and hinders the agent's long-term planning capacity. For example, at step 7, the agent seems to realize that it is navigating in the wrong direction and intends to backtrack. However, after completing a single-step backtracking at step 8, it again decides to go back to the wrong place it was at step 7. This behavior of oscillating between two locations persists until navigation failure at step 15. On the other hand, the global planning space enables the agent to capture global environment layouts and successfully correct previous wrong decisions. At step 4, the agent also starts by navigating in the wrong direction, just like the local planning variants. But the predicted long-term goal effectively guides it back on the right track at step 8, concluding with successful navigation.

As shown in Fig. 6, the practical sliding-forbidden steps can cause the agent to get stuck in obstacles and lead to navigation failure. For instance, in the absence of Tryout, the agent is unable to proceed forward once its chassis collides with the wall (at steps 6 and 7). This situation persists until the end of navigation (at step 14), where the agent does not succeed in escaping the deadlock, ultimately leading to navigation failure. Conversely, the integration of Tryout control in our model effectively addresses this issue. At step 4, Tryout is triggered upon colliding with the wall, causing the agent to twist and stagger away from the obstacle. This helps to navigate around the obstacle, with the navigation accomplished at step 6 successfully.

V. CONCLUSION

In summary, this paper introduces ETPNav, a novel navigation system that leverages topological maps for VLN-CE. We first propose an online mapping method via waypoint self-organization to enable robust long-range planning of an agent. This scheme doesn't require any prior environmental experience and satisfies the demand of navigation in realistic scenarios. Then, we systematically examine our topo map's key design

choices and empirically show that a concise depth-only design can be optimal for waypoint prediction. Furthermore, we address an often-neglected issue in VLN-CE - obstacle avoidance, with a simple and effective trial-and-error controller. Extensive experiments demonstrate the effectiveness of the proposed method, yielding more than 10% and 20% absolute improvements over prior state-of-the-art on R2R-CE and RxR-CE benchmarks, respectively. We hope this work can serve as a strong baseline for further research on this challenging task.

Future Work: ETPNav employs pose-reading sensors to facilitate topological mapping. However, in real-world applications, the presence of sensor actuation noise may introduce complexities into the mapping process. We take the noise analysis and robustness study in such scenarios as the future work plan.

REFERENCES

- [1] P. Anderson et al., "Vision-and-language navigation: Interpreting visually-grounded navigation instructions in real environments," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 3674–3683.
- [2] X. Wang et al., "Reinforced cross-modal matching and self-supervised imitation learning for vision-language navigation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 6629–6638.
- [3] H. Wang, W. Wang, W. Liang, C. Xiong, and J. Shen, "Structured scene memory for vision-language navigation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 8455–8464.
- [4] Y. Hong, Q. Wu, Y. Qi, C. Rodriguez-Opazo, and S. Gould, "VLN BERT: A recurrent vision-and-language BERT for navigation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 1643–1653.
- [5] S. Chen, P.-L. Guhur, C. Schmid, and I. Laptev, "History aware multimodal transformer for vision-and-language navigation," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 5834–5847.
- [6] J. Krantz, E. Wijmans, A. Majumdar, D. Batra, and S. Lee, "Beyond the NAV-graph: Vision-and-language navigation in continuous environments," in *Proc. Eur. Conf. Comput. Vis.*, Springer, 2020, pp. 104–120.
- [7] S. Raychaudhuri, S. Wani, S. Patel, U. Jain, and A. Chang, "Language-aligned waypoint (LAW) supervision for vision-and-language navigation in continuous environments," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2021, pp. 4018–4028.
- [8] J. Krantz, A. Gokaslan, D. Batra, S. Lee, and O. Maksymets, "Waypoint models for instruction-guided navigation in continuous environments," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2021, pp. 15162–15171.
- [9] Y. Hong, Z. Wang, Q. Wu, and S. Gould, "Bridging the gap between learning in discrete and continuous environments for vision-and-language navigation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 15439–15449.
- [10] J. Krantz and S. Lee, "Sim-2-Sim transfer for vision-and-language navigation in continuous environments," in *Proc. Eur. Conf. Comput. Vis.*, Springer, 2022, pp. 588–603.
- [11] D. An et al., "1st place solutions for RxR-habitat vision-and-language navigation competition," 2022, *arXiv:2206.11610*.
- [12] J. A. Sethian, "A fast marching level set method for monotonically advancing fronts," *Proc. Nat. Acad. Sci.*, vol. 93, no. 4, pp. 1591–1595, 1996.
- [13] E. Wijmans et al., "DD-PPO: Learning near-perfect pointgoal navigators from 2.5 billion frames," in *Proc. Int. Conf. Learn. Representations*, 2019.
- [14] G. Georgakis et al., "Cross-modal map learning for vision and language navigation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 15460–15470.
- [15] S. B. Udin and J. W. Fawcett, "Formation of topographic maps," *Annu. Rev. Neurosci.*, vol. 11, no. 1, pp. 289–327, 1988.
- [16] Z. Deng, K. Narasimhan, and O. Russakovsky, "Evolving graphical planner: Contextual global planning for vision-and-language navigation," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 20660–20672.
- [17] S. Chen, P.-L. Guhur, M. Tapaswi, C. Schmid, and I. Laptev, "Think global, act local: Dual-scale graph transformer for vision-and-language navigation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 16537–16547.

- [18] K. Chen, J. K. Chen, J. Chuang, M. Vázquez, and S. Savarese, "Topological planning with transformers for vision-and-language navigation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 11276–11286.
- [19] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 5998–6008.
- [20] P. Xu, X. Zhu, and D. A. Clifton, "Multimodal learning with transformers: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 10, pp. 12113–12132, Oct. 2023.
- [21] M. Deitke et al., "Retrospectives on the embodied AI workshop," 2022, *arXiv:2210.06849*.
- [22] A. Ku, P. Anderson, R. Patel, E. Ie, and J. Baldridge, "Room-across-room: Multilingual vision-and-language navigation with dense spatiotemporal grounding," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2020, pp. 4392–4412.
- [23] H. Chen, A. Suhr, D. Misra, N. Snively, and Y. Artzi, "Touchdown: Natural language navigation and spatial reasoning in visual street environments," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 12538–12547.
- [24] J. Thomason, M. Murray, M. Cakmak, and L. Zettlemoyer, "Vision-and-dialog navigation," in *Proc. Conf. Robot Learn.*, PMLR, 2020, pp. 394–406.
- [25] K. Nguyen and H. DauméIII, "Help, anna! visual navigation with natural multimodal assistance via retrospective curiosity-encouraging imitation learning," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2019, pp. 684–695.
- [26] Y. Qi et al., "Reverie: Remote embodied visual referring expression in real indoor environments," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 9982–9991.
- [27] F. Zhu, X. Liang, Y. Zhu, Q. Yu, X. Chang, and X. Liang, "Soon: Scenario oriented object navigation with graph-based exploration," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 12689–12699.
- [28] D. Fried et al., "Speaker-follower models for vision-and-language navigation," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 3318–3329.
- [29] C.-Y. Ma et al., "Self-monitoring navigation agent via auxiliary progress estimation," in *Proc. Int. Conf. Learn. Representations*, 2019.
- [30] Y. Qi, Z. Pan, S. Zhang, A. v. d. Hengel, and Q. Wu, "Object-and-action aware model for visual language navigation," in *Proc. Eur. Conf. Comput. Vis.*, Springer, 2020, pp. 303–317.
- [31] Y. Hong, C. Rodriguez, Y. Qi, Q. Wu, and S. Gould, "Language and visual entity relationship graph for agent navigation," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 7685–7696.
- [32] D. An, Y. Qi, Y. Huang, Q. Wu, L. Wang, and T. Tan, "Neighbor-view enhanced model for vision and language navigation," in *Proc. ACM Int. Conf. Multimedia*, 2021, pp. 5101–5109.
- [33] X. Wang, W. Xiong, H. Wang, and W. Y. Wang, "Look before you leap: Bridging model-free and model-based reinforcement learning for planned-ahead vision-and-language navigation," in *Proc. Eur. Conf. Comput. Vis.*, 2018, pp. 37–53.
- [34] H. Tan, L. Yu, and M. Bansal, "Learning to navigate unseen environments: Back translation with environmental dropout," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics*, 2019, pp. 2610–2621.
- [35] H. Wang, Q. Wu, and C. Shen, "Soft expert reward learning for vision-and-language navigation," in *Proc. Eur. Conf. Comput. Vis.*, Springer, 2020, pp. 126–141.
- [36] H. Wang, W. Wang, T. Shu, W. Liang, and J. Shen, "Active visual information gathering for vision-language navigation," in *Proc. Eur. Conf. Comput. Vis.*, Springer, 2020, pp. 307–322.
- [37] H. Wang, W. Wang, W. Liang, S. C. Hoi, J. Shen, and L. V. Gool, "Active perception for visual-language navigation," *Int. J. Comput. Vis.*, vol. 131, no. 3, pp. 607–625, 2023.
- [38] C. Liu, F. Zhu, X. Chang, X. Liang, Z. Ge, and Y.-D. Shen, "Vision-language navigation with random environmental mixup," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2021, pp. 1644–1654.
- [39] J. Li, H. Tan, and M. Bansal, "EnvEdit: Environment editing for vision-and-language navigation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 15407–15417.
- [40] A. Parvaneh, E. Abbasnejad, D. Teney, J. Q. Shi, and A. van denHengel, "Counterfactual vision-and-language navigation: Unravelling the unseen," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 5296–5307.
- [41] A. Kamath et al., "A new path: Scaling vision-and-language navigation with synthetic instructions and imitation learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 10813–10823.
- [42] H. Wang, W. Liang, J. Shen, L. Van Gool, and W. Wang, "Counterfactual cycle-consistent learning for instruction following and generation in vision-language navigation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 15471–15481.
- [43] S. Wang et al., "Less is more: Generating grounded navigation instructions from landmarks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 15428–15438.
- [44] M. Zhao et al., "On the evaluation of vision-and-language navigation instructions," in *Proc. Conf. Eur. Chapter Assoc. Comput. Linguistics*, 2021, pp. 1302–1316.
- [45] X. Wang, W. Wang, J. Shao, and Y. Yang, "LANA: A language-capable navigator for instruction dreaming and generation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 19048–19058.
- [46] X. Wang, W. Wang, and J. Shao, "Learning to follow and generate instructions for language-capable navigation," *IEEE Trans. Pattern Anal. Mach. Intell.*, pp. 1–17, 2023.
- [47] W. Hao, C. Li, X. Li, L. Carin, and J. Gao, "Towards learning a generic agent for vision-and-language navigation via pre-training," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 13137–13146.
- [48] P.-L. Guhur, M. Tapaswi, S. Chen, I. Laptev, and C. Schmid, "AirBERT: In-domain pretraining for vision-and-language navigation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2021, pp. 1634–1643.
- [49] A. Majumdar, A. Shrivastava, S. Lee, P. Anderson, D. Parikh, and D. Batra, "Improving vision-and-language navigation with image-text pairs from the web," in *Proc. Eur. Conf. Comput. Vis.*, Springer, 2020, pp. 259–274.
- [50] Y. Qi, Z. Pan, Y. Hong, M.-H. Yang, A. van den Hengel, and Q. Wu, "The road to know-where: An object-and-room informed sequential BERT for indoor vision-language navigation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2021, pp. 1655–1664.
- [51] A. Moudgil, A. Majumdar, H. Agrawal, S. Lee, and D. Batra, "Soat: A scene-and-object-aware transformer for vision-and-language navigation," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 7357–7367.
- [52] Y. Qiao, Y. Qi, Y. Hong, Z. Yu, P. Wang, and Q. Wu, "HOP : History-enhanced and order-aware pre-training for vision-and-language navigation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 7, pp. 8524–8537, Jul. 2023.
- [53] C. Lin, Y. Jiang, J. Cai, L. Qu, G. Haffari, and Z. Yuan, "Multimodal transformer with variable-length memory for vision-and-language navigation," in *Proc. Eur. Conf. Comput. Vis.*, Springer, 2022, pp. 380–397.
- [54] Y. Zhao et al., "Target-driven structured transformer planner for vision-language navigation," in *Proc. ACM Int. Conf. Multimedia*, 2022, pp. 4194–4203.
- [55] M. Hwang, J. Jeong, M. Kim, Y. Oh, and S. Oh, "Meta-explore: Exploratory hierarchical vision-and-language navigation using scene object spectrum grounding," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 6683–6693.
- [56] R. Liu, X. Wang, W. Wang, and Y. Yang, "Bird's-eye-view scene graph for vision-language navigation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2023, pp. 10968–10980.
- [57] D. An et al., "BEVBert: Multimodal map pre-training for language-guided navigation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2023, pp. 2737–2748.
- [58] B. Lin, Y. Zhu, Z. Chen, X. Liang, J. Liu, and X. Liang, "Adapt: Vision-language navigation with modality-aligned action prompts," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 15396–15406.
- [59] X. Liang, F. Zhu, L. Lingling, H. Xu, and X. Liang, "Visual-language navigation pretraining via prompt-based environmental self-exploration," in *Proc. Annu. Meeting Assoc. Comput. Linguistics*, 2022, pp. 4837–4851.
- [60] P. Anderson et al., "Sim-to-real transfer for vision-and-language navigation," in *Proc. Conf. Robot Learn.*, PMLR, 2021, pp. 671–681.
- [61] M. Savva et al., "Habitat: A platform for embodied AI research," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 9339–9347.
- [62] M. Z. Irshad, N. C. Mithun, Z. Seymour, H.-P. Chiu, S. Samarasekera, and R. Kumar, "SASRA: Semantically-aware spatio-temporal reasoning agent for vision-and-language navigation in continuous environments," in *Proc. Int. Conf. Pattern Recognit.*, 2022, pp. 4065–4071.
- [63] P. Chen et al., "Weakly-supervised multi-granularity map learning for vision-and-language navigation," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 38149–38161.
- [64] H. Wang, W. Liang, L. V. Gool, and W. Wang, "Dreamwalker: Mental planning for continuous vision-language navigation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2023, pp. 10873–10883.
- [65] A. Elfes, "Using occupancy grids for mobile robot perception and navigation," *Computer*, vol. 22, no. 6, pp. 46–57, 1989.
- [66] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardos, "ORB-SLAM: A versatile and accurate monocular slam system," *IEEE Trans. Robot.*, vol. 31, no. 5, pp. 1147–1163, Oct. 2015.

- [67] R. A. Newcombe, S. J. Lovegrove, and A. J. Davison, "DTAM: Dense tracking and mapping in real-time," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2011, pp. 2320–2327.
- [68] F. Dellaert, D. Fox, W. Burgard, and S. Thrun, "Monte Carlo localization for mobile robots," in *Proc. Int. Conf. Robot. Automat.*, 1999, pp. 1322–1328.
- [69] J. F. Henriques and A. Vedaldi, "MapNet: An allocentric spatial memory for mapping environments," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 8476–8484.
- [70] D. S. Chaplot, M. Dalal, S. Gupta, J. Malik, and R. R. Salakhutdinov, "SEAL: Self-supervised embodied active learning using exploration and 3D consistency," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, pp. 13086–13098, 2021.
- [71] D. S. Chaplot, D. Gandhi, S. Gupta, A. Gupta, and R. Salakhutdinov, "Learning to explore using active neural SLAM," in *Proc. Int. Conf. Learn. Representations*, 2019.
- [72] D. S. Chaplot, D. P. Gandhi, A. Gupta, and R. R. Salakhutdinov, "Object goal navigation using goal-oriented semantic exploration," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 4247–4258.
- [73] T. Gervet, S. Chintala, D. Batra, J. Malik, and D. S. Chaplot, "Navigating to objects in the real world," *Sci. Robot.*, vol. 8, no. 79, 2023, Art. no. eadf 6991.
- [74] S. Y. Min, D. S. Chaplot, P. K. Ravikumar, Y. Bisk, and R. Salakhutdinov, "Film: Following instructions in language with modular methods," in *Proc. Int. Conf. Learn. Representations*, 2021.
- [75] J. Chen, W. Wang, S. Liu, H. Li, and Y. Yang, "Omnidirectional information gathering for knowledge transfer-based audio-visual navigation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2023, pp. 10993–11003.
- [76] H. Choset and K. Nagatani, "Topological simultaneous localization and mapping (SLAM): Toward exact localization without explicit localization," *IEEE Trans. Robot. Automat.*, vol. 17, no. 2, pp. 125–137, Apr. 2001.
- [77] B. Kuipers and Y.-T. Byun, "A robot exploration and mapping strategy based on a semantic hierarchy of spatial representations," *Robot. Auton. Syst.*, vol. 8, no. 1/2, pp. 47–63, 1991.
- [78] D. S. Chaplot, R. Salakhutdinov, A. Gupta, and S. Gupta, "Neural topological SLAM for visual navigation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 12875–12884.
- [79] O. Kwon, N. Kim, Y. Choi, H. Yoo, J. Park, and S. Oh, "Visual graph memory with unsupervised representation for visual navigation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2021, pp. 15890–15899.
- [80] M. Hahn, D. S. Chaplot, S. Tulsiani, M. Mukadam, J. M. Rehg, and A. Gupta, "No RL, no simulation: Learning to navigate without navigating," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 26661–26673.
- [81] A. Chang et al., "Matterport3D: Learning from RGB-D data in indoor environments," in *Proc. Int. Conf. 3D Vis.*, 2017, pp. 667–676.
- [82] X. Zhao, H. Agrawal, D. Batra, and A. G. Schwing, "The surprising effectiveness of visual odometry techniques for embodied point-goal navigation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2021, pp. 16127–16136.
- [83] J. D.M.-W. C. Kenton and L. K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics*, 2019, pp. 4171–4186.
- [84] H. Tan and M. Bansal, "Lxmert: Learning cross-modality encoder representations from transformers," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2019, pp. 5100–5111.
- [85] H. Luo, A. Yue, Z.-W. Hong, and P. Agrawal, "Stubborn: A strong baseline for indoor object navigation," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2022, pp. 3287–3293.
- [86] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proc. 14th Int. Conf. Artif. Intell. Statist. JMLR Workshop Conf. Proc.*, 2011, pp. 627–635.
- [87] P. Anderson et al., "On evaluation of embodied navigation agents," 2018, *arXiv: 1807.06757*.
- [88] G. Ilharco, V. Jain, A. Ku, E. Ie, and J. Baldridge, "General evaluation for instruction conditioned navigation using dynamic time warping," 2019, *arXiv: 1907.05446*.
- [89] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. Int. Conf. Learn. Representations*, 2020.
- [90] A. Radford et al., "Learning transferable visual models from natural language supervision," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2021, pp. 8748–8763.
- [91] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.
- [92] Y. Liu et al., "RoBERTa: A robustly optimized BERT pretraining approach," 2019, *arXiv: 1907.11692*.
- [93] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, pp. 8024–8035, 2019.
- [94] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. Int. Conf. Learn. Representations*, 2019.
- [95] S. Bengio, O. Vinyals, N. Jaitly, and N. Shazeer, "Scheduled sampling for sequence prediction with recurrent neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2015, pp. 1171–1179.



place at CVPR 2022 RxR-Habitat Vision-Language Navigation Challenge and 2nd place at CSIG 2022 REVERIE Vision-Language Navigation Challenge.



His current research interests include deep reinforcement learning and computer vision.



Australia. He has published more than 80 papers in leading international journals and conferences such as TPAMI, ICML, ICLR, NeurIPS, CVPR, and ICCV. His current research interests include neuro-symbolic AI, human-centric AI, and embodied AI. He is an associate editor of *IEEE Transactions on Circuits and Systems for Video Technology* and *Neurocomputing*.



Vision-Language Navigation Challenge.

Dong An received the bachelor's degree in computer science from Peking University (PKU), Beijing, China, in 2019. He is currently working toward the PhD degree with the Institute of Automation, Chinese Academy of Sciences (CASIA), and the University of Chinese Academy of Sciences (UCAS), Beijing, China. His research interests include computer vision and pattern recognition, with a focus on Embodied AI and Multimodal Learning. He has published several papers in top-level international conferences, such as ICCV, NeurIPS, ACM MM, and BMVC. He won 1st place at CVPR 2022 RxR-Habitat Vision-Language Navigation Challenge and 2nd place at CSIG 2022 REVERIE Vision-Language Navigation Challenge.

Hanqing Wang is working toward the PhD degree with the Media Computing and Intelligent Systems Lab, Beijing Institute of Technology, advised by Prof. Wei Liang and Prof. Mintao Pei. In 2017, he did his internship with Microsoft Research Asia, VC Group, mentored by research scientist Jialong Yang. From 2019 to 2020, he did his internship with Inception Institute of Artificial Intelligence, UAE, advised by Dr. Wenguan Wang and Prof. Jianbing Shen. Currently, he did internship with ETH Zurich, Switzerland, advised by Dr. Wenguan Wang and Prof. Luc Van Gool.

Wenguan Wang (Senior Member, IEEE) received the PhD degree from the Beijing Institute of Technology, in 2018. He is currently a ZJU100 young professor with Zhejiang University, China. From 2016 to 2018, he was a visiting PhD candidate with the University of California, Los Angeles, USA. From 2018 to 2019, he was a senior scientist with the Inception Institute of Artificial Intelligence, UAE. From 2020 to 2022, he was a research fellow with ETH Zurich, Switzerland. From 2022–2023, he was a lecturer and DECRA fellow with the University of Technology Sydney,

Zun Wang received the bachelor's degree in mathematics from the University of Science and Technology of China. He is currently working toward the master's degree with the Australian National University, advised by Prof. Stephen Gould. His current research interest lies in Embodied AI, especially in Vision-Language Navigation. He has published several papers regarding Vision-Language Navigation at international conferences, such as CVPR, ICCV. He won the CVPR 2022 RxR-Habitat Vision-Language Navigation Challenge and the CSIG 2022 REVERIE



Yan Huang is currently an associate professor with the Institute of Automation, Chinese Academy of Sciences (CASIA). His research interests include computer vision and cross-modal data analysis. He has published more than 80 papers in the leading international journals and conferences such as the *IEEE Transactions on Pattern Analysis and Machine Intelligence*, NeurIPS, CVPR, and ICCV. He has obtained awards such as the Presidential Special Award of CAS, Excellent Doctoral Thesis of both CAS and CAAI, NVIDIA Pioneering Research Award, Baidu Fellowship, CVPR 2014 Workshop Best Paper Award, ICPR 2014 Best Student Paper Award, and RACV 2016 Best Poster Award. He has served as a co-chair of ICCV 2019 Workshop on Cross-Modal Learning in Real World and CVPR 2020 Workshop on Multimodal Learning.



search interests include multimodal learning and machine learning.

Keji He received the BSc degree from the Nanjing University of Posts and Telecommunications (NJUPT), Nanjing, China, in 2019. He is currently working toward the PhD degree with the Institute of Automation, Chinese Academy of Sciences (CASIA), Beijing, China, and the University of Chinese Academy of Sciences (UCAS), Beijing, China. He is also currently a joint PhD in National University of Singapore (NUS). He published papers in top-level conferences and journals, such as NeurIPS and JAS. He has served as a reviewer of ACM MM. His research interests include multimodal learning and machine learning.



Liang Wang (Fellow, IEEE) received the BEng and MEng degrees from Anhui University, in 1997 and 2000, respectively, and the PhD degree from the Institute of Automation, Chinese Academy of Sciences (CASIA), in 2004. From 2004 to 2010, he was a research assistant with Imperial College London, U.K., and Monash University, Australia, a research fellow with the University of Melbourne, Australia, and a lecturer with the University of Bath, U.K., respectively. Currently, he is a full professor of the Hundred Talents Program with the National Lab of Pattern Recognition, CASIA. His major research interests include machine learning, pattern recognition, and computer vision. He has widely published in highly ranked international journals, such as the *IEEE Transactions on Pattern Analysis and Machine Intelligence* and *IEEE Transactions on Image Processing*, and leading international conferences, such as CVPR, ICCV, and ECCV. He has served as an associate editor of the *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *IEEE Transactions on Image Processing*, and *Pattern Recognition*. He is an IAPR fellow.

Liang Wang (Fellow, IEEE) received the BEng and MEng degrees from Anhui University, in 1997 and 2000, respectively, and the PhD degree from the Institute of Automation, Chinese Academy of Sciences (CASIA), in 2004. From 2004 to 2010, he was a research assistant with Imperial College London, U.K., and Monash University, Australia, a research fellow with the University of Melbourne, Australia, and a lecturer with the University of Bath, U.K., respectively. Currently, he is a full professor of the Hundred Talents Program with the National Lab of

Pattern Recognition, CASIA. His major research interests include machine learning, pattern recognition, and computer vision. He has widely published in highly ranked international journals, such as the *IEEE Transactions on Pattern Analysis and Machine Intelligence* and *IEEE Transactions on Image Processing*, and leading international conferences, such as CVPR, ICCV, and ECCV. He has served as an associate editor of the *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *IEEE Transactions on Image Processing*, and *Pattern Recognition*. He is an IAPR fellow.